

Architectural Guidelines and Code Patterns for a Hybrid LangGraph and CrewAI Lead Investigator Node

1. Macro-Architectural Paradigm: The Convergence of Deterministic Orchestration and Autonomous Agents

The enterprise deployment of artificial intelligence has transitioned from experimental, single-prompt conversational interfaces to complex, multi-agent systems designed to execute long-running, stateful workflows. As organizations seek to automate intricate processes such as Business-to-Business (B2B) lead generation, human resources contact discovery, and executive research, the limitations of monolithic language model invocations become starkly apparent.¹ A highly specialized system is required to navigate corporate hierarchies, leverage Open-Source Intelligence (OSINT) tools, and validate contact data. To achieve this, the system architecture must bridge two distinct paradigms: the rigid, deterministic control flow of graph-based state machines and the dynamic, role-playing autonomy of multi-agent collaborations.⁴

LangGraph and CrewAI represent the zenith of these respective paradigms. LangGraph operates on the fundamental principle of stateful orchestration, modeling workflows as directed graphs where individual Python functions act as nodes, and conditional logic dictates the traversal edges.⁶ This explicit graph-based Domain-Specific Language (DSL) offers granular control over state mutations, dependency management, and persistent memory across distributed executions.⁴ However, encoding complex cognitive tasks—such as dynamically determining which combination of web scraping and email verification tools to employ based on ambiguous search results—into rigid graph nodes often leads to brittle, overly complex state machines.⁷

Conversely, CrewAI abstracts orchestration through a role-based, multi-agent collaborative model.⁸ Within CrewAI, developers define specialized personas complete with backstories, goals, and specific toolsets, allowing the underlying language model to autonomously determine the execution path required to fulfill a given task.¹⁰ This event-driven, implicit orchestration vastly accelerates the development of reasoning-heavy operations.⁴ Nevertheless, in massive enterprise applications involving hundreds of target companies, relying solely on CrewAI can introduce unpredictability; delegation chains may become fragile during long-running tasks, and isolating fault boundaries for automated retries becomes challenging.⁵

The architectural blueprint detailed in this document proposes a hybrid approach: nesting a CrewAI micro-executor within a LangGraph macro-orchestrator node.¹² By wrapping the

CrewAI execution environment—specifically the `crew.kickoff()` invocation—inside a distinct LangGraph node function, the system encapsulates the non-linear reasoning of multi-agent dialogue within a strictly typed, version-controlled, and checkpoint-backed state machine.⁶ This hybrid "Lead Investigator" node ensures that the overarching application logic, error recovery, and human-in-the-loop interventions are managed deterministically by LangGraph, while the nuanced, heuristic-heavy data discovery is delegated to CrewAI's autonomous agents. This document provides the comprehensive context, state schemas, code patterns, and prompt engineering directives required for an advanced language model to synthesize the final Python implementation of this system.

2. Global State Management and Pydantic Schema Design

The structural integrity of a LangGraph application relies entirely on its shared state object. Every node in the directed graph receives the complete state upon invocation and returns a partial update containing only the localized state mutations it has executed.⁶ LangGraph subsequently merges these updates via user-defined reducer functions and serializes the state to a persistent storage backend, such as SQLite or PostgreSQL, establishing a checkpoint before transitioning to the subsequent node.¹

2.1 Bridging the Macro and Micro Contexts

In the hybrid architecture, the global state serves a dual purpose. It must maintain the macro-orchestration variables necessary for LangGraph's conditional routing, and it must simultaneously harbor the seed data required to instantiate the CrewAI tasks dynamically.¹³ Translating the LangGraph state into actionable CrewAI inputs requires a meticulous mapping strategy, shifting from the traditional TypedDict conventions of LangGraph to the heavily validated Pydantic BaseModel structures preferred by modern agentic frameworks.⁷

The global state schema must be engineered to prevent data overwrites during cyclical execution. For a B2B contact discovery pipeline, the state will aggregate findings across multiple target organizations.¹

State Attribute	Structural Type	Macro-Orchestrator or Function (LangGraph)	Micro-Executor Integration (CrewAI)
messages	Annotated, add_messages]	Maintains the comprehensive execution trace and conversation	Appended with the finalized, structured findings generated by the CrewAI

		history across all graph transitions. ¹³	agent execution. ¹²
target_companies	List[dict]	Stores the backlog of corporate entities identified by upstream discovery nodes (e.g., a Job Hunter agent). ¹	Passed sequentially or concurrently as dynamic inputs to the CrewAI kickoff payload. ¹²
investigation_results	Annotated, operator.add]	Aggregates all successfully verified corporate contacts, enriched with metadata and confidence scores. ¹	Serves as the strictly typed Pydantic output target for the CrewAI task. ¹⁵
escalation_flag	bool	Serves as the trigger mechanism for conditional edges, routing execution to a Human-in-the-Loop approval node. ¹	Evaluated based on the confidence thresholds or empty outputs returned by the CrewAI framework. ¹
system_faults	Annotated[List[str], operator.add]	An append-only ledger of unhandled exceptions, API timeouts, or fatal parsing errors.	Captured and reported if the encapsulated crew.kickoff() execution throws a cascading error. ¹²

2.2 Enforcing Structured Task Outputs

A critical vulnerability in traditional multi-agent systems is the reliance on unstructured text parsing to extract actionable data from the language model's final response.¹⁸ Utilizing secondary nodes or heuristic regular expressions to convert conversational text into JSON payloads doubles network latency, escalates token consumption, and introduces severe architectural fragility.¹⁹

To seamlessly interface the CrewAI micro-executor back into the LangGraph state, the architecture mandates the enforcement of structured data extraction directly at the task

execution level. CrewAI facilitates this through the `output_pydantic` configuration parameter within its Task class.¹⁶

By defining a comprehensive Pydantic model representing the expected B2B lead—for example, a schema incorporating fields for the contact's full name, corporate title, verified email address, source URL, and an algorithmic confidence score—the system drastically reduces integration complexity. When `output_pydantic` is assigned to a task, CrewAI's underlying engine automatically instructs the associated `BaseAgentAdapter` on the expected format.¹⁵ The framework actively modifies the agent's system prompt to inject rigorous schema compliance instructions and subsequently executes internal validation and post-processing on the raw language model output.¹⁵ Consequently, the `CrewOutput` object returned to the LangGraph node wrapper exposes a fully validated Python object, which can be directly appended to the `investigation_results` state variable without intermediate parsing logic.¹⁶

2.3 Designing Flexible Schemas for Local Inference

When designing these Pydantic schemas, particularly for systems anticipating the use of localized Open-Weight models (such as Llama 3 or Qwen) rather than frontier commercial APIs, schema flexibility is paramount. Local models often exhibit degraded performance when confronted with overly rigid constraints on deeply nested JSON objects, sometimes hallucinating data simply to satisfy a required field that could not be legitimately discovered.¹⁸

The architectural best practice is to mandate the inclusion of the Python typing.Optional wrapper for all attributes that cannot be guaranteed during a web scraping or OSINT operation. By allowing fields like `linkedin_profile_url` or `direct_phone_number` to accept null values, the schema maintains backward compatibility and prevents the model from generating fictitious data points, thereby preserving the integrity of the B2B contact dataset.¹⁸

3. Context Engineering and the Lead Investigator Persona

Within the CrewAI framework, the behavioral boundaries and heuristic methodologies of the language model are governed entirely by its assigned persona.²² While the mechanical task definition provides the explicit operational goal, the persona dictates the agent's critical thinking protocols, error recovery strategies, and interaction style.²³

3.1 Establishing the Role, Goal, and Backstory Triad

Deploying generic agent definitions inevitably results in generic, superficial outputs.²³ A Lead Investigator tasked with uncovering high-level decision-makers must operate with a highly calibrated, specialized persona designed to navigate corporate obfuscation, parse 10-K financial reports, and interpret complex corporate alignment decks.³ The foundation of this persona relies on the Role, Goal, and Backstory triad.¹⁰

The Role attribute defines the agent's precise function and systemic expertise.¹⁰ For this node, assigning a role such as "Senior Open-Source Intelligence (OSINT) B2B Contact Strategist" immediately grounds the language model in a specific professional domain, activating the latent weights associated with corporate research and data enrichment.²

The Goal parameter provides the unyielding objective guiding the agent's autonomous decision-making loops.¹⁰ This must be exceedingly precise; a vague goal such as "find email addresses" encourages the agent to terminate its search prematurely upon finding generic info@ or support@ addresses. The operational goal must be articulated as identifying, cryptographically verifying, and compiling the direct contact information for senior HR decision-makers, technical recruiting managers, and talent acquisition directors at the specified target organizations.¹

The Backstory attribute is the most vital component for shaping the agent's analytical approach and operational values.²³ An effective backstory for the Lead Investigator must instill a prioritization of data accuracy over execution speed. The narrative should establish that the agent possesses a decade of experience in executive search, understands the intricate nuances of corporate email conventions (e.g., permutations of first name, last name, and domain), and harbors a meticulous methodology for cross-referencing public directories against automated domain scraping outputs.²³ By explicitly stating in the backstory that the agent refuses to guess or fabricate contact details without multi-source validation, the system establishes a programmatic safeguard against hallucination.²³

3.2 Advanced Prompt and Context Engineering Tactics

Translating this persona into a high-performance system necessitates sophisticated prompt engineering techniques.²⁵ High-quality generative results in a production B2B environment require precise alignment with business policies, clear definitions of expected outputs, and robust mechanisms for handling task complexity.²⁵ The core of this discipline is context engineering—the process of structuring the immediate situational data surrounding the prompt.²⁶

Context engineering dictates that the agent must be explicitly aware of the broader organizational objective.²⁷ For instance, providing the contextual background that the discovery operation is aimed at placing senior machine learning engineers informs the agent's search parameters, subtly guiding its heuristics away from campus recruiters and toward senior technical talent acquisition partners.¹

Furthermore, as the agent operates within its autonomous execution loop, generating tool calls, reading web pages, and evaluating errors, the conversational context window rapidly expands.²⁸ Without active context management, the language model's attention mechanism degrades, leading to task amnesia or repetitive tool invocation.²⁸ Advanced architectural implementations utilize context compaction strategies. The system prompt must instruct the agent to continuously distill its findings, discarding the raw HTML payloads of redundant tool

outputs while preserving critical facts, architectural decisions, and verified data points in a high-fidelity summarized format.²⁸ This ensures that the agent maintains operational continuity throughout lengthy investigations without breaching context limits or incurring exorbitant token costs.²⁸

Finally, the prompt engineering strategy must rigidly dictate the output tone.²⁷ The Lead Investigator is a programmatic component feeding data into a broader orchestration graph; conversational pleasantries, introductory phrasing, and explanatory filler are entirely prohibited. The prompt must force an exclusively analytical, objective, and formatting-compliant response tone, ensuring that the final output aligns perfectly with the expected Pydantic serialization structure.²⁷

4. Capability Augmentation and Tool Integration

An autonomous agent isolated from external data sources is fundamentally incapable of executing B2B research. CrewAI extends its core language model capabilities through five distinct integration vectors: callable Tools, Model Context Protocol (MCP) servers, platform Apps, injected Skills, and vector-backed Knowledge.²⁹

4.1 Architecting the Discovery Arsenal

The Lead Investigator node relies entirely on its action capabilities—specifically Tools and MCPs—to manipulate the external environment and retrieve authoritative data.²⁹ At execution time, these distinct integration vectors are unified internally into BaseTool instances, presenting the agent with a cohesive catalog of executable functions.²⁹

To fulfill the lead generation and enrichment mandate, the agent must be equipped with a comprehensive discovery arsenal.²⁴ The primary requirement is a web search and extraction utility, such as a localized DuckDuckGo integration or a dedicated scraping API wrapper (e.g., Firecrawl), enabling the agent to autonomously navigate corporate "About Us" pages, scan press releases, and parse digital directories for leadership hierarchies.¹¹

Crucially, the discovery process must be augmented with cryptographic validation tools. The agent must possess the capability to interface with domain verification APIs (e.g., Hunter.io or Snov.io) to query standard corporate email structures and execute non-intrusive SMTP ping validations.¹ This toolset empowers the agent to transition from merely guessing an email based on a name to mathematically verifying its existence before appending it to the LangGraph state.

For advanced enterprise deployments, the toolset should be further expanded through App integrations to connect directly with the organization's Customer Relationship Management (CRM) platform.²⁹ By providing a tool that queries Salesforce or HubSpot, the Lead Investigator can cross-reference newly discovered leads against historical outreach campaigns, automatically filtering out existing contacts and preventing redundant sales engagements.²

4.2 Overcoming the Complexities of Local LLM Tool Calling

While integrating frontier commercial models (such as GPT-4o or Claude 3.5 Sonnet) offers out-of-the-box reliability for complex tool calling, deploying the architecture utilizing localized, open-weight models fundamentally alters the engineering requirements. Implementing the LangGraph and CrewAI stack atop a local inference engine like Ollama (running models such as Llama 3, Qwen 2.5, or Mistral) guarantees absolute data privacy and eliminates recurring API costs, which is highly advantageous for scraping sensitive corporate data.¹ However, local models demonstrate severe inconsistencies when navigating autonomous tool-calling loops.³⁰

Extensive diagnostic evaluations of local models operating within multi-agent frameworks reveal critical failure modes.³¹ Common anomalies include hallucinating non-existent tool names (e.g., inventing a `calculate_total_files` function instead of selecting from the provided list), becoming trapped in infinite execution loops where the agent repeatedly calls the same function without analyzing the output, or failing to format the required JSON arguments correctly.³¹ Furthermore, routing mechanisms like LiteLLM may inadvertently strip tool definitions if the local endpoint is misidentified as not supporting native function calling.³⁵

To forge a reliable Lead Investigator using Ollama, developers must abandon standard zero-shot prompts and implement aggressive, hyper-constrained prompt engineering protocols.³⁶

First, inference stochasticity must be eliminated. The model's generation temperature must be explicitly locked to 0.0. Tool calling is fundamentally a deterministic syntax generation task; any probabilistic sampling variance introduces a high probability of malformed JSON structures or creative, yet fatal, deviations from the provided tool schemas.¹⁸

Second, the system prompt injected into the local model must transition from a passive suggestion to an absolute directive.³⁷ The orchestrator must force the model to adhere strictly to a predefined action schema. An effective architectural pattern involves embedding a rigid JSON template directly into the system instructions. The model must be commanded that its sole purpose is to output a single JSON object corresponding to the following structure: `{"action": "call_tool" | "finish", "target": string, "arguments": object, "reason": string}`.³⁷ The prompt must explicitly forbid the generation of conversational text outside of this JSON payload and mandate that the "finish" action is only invoked when the overarching goal has been definitively met.³⁷

Third, developers working with quantized, sub-14-billion parameter models (such as Qwen2.5:7b or Llama3:8b) must implement robust programmatic feedback loops to sanitize the model's output.³² Rather than trusting the model to execute the tool directly, the system should employ regular expressions to parse the raw text stream, extract the tool designation and parameters, validate them against the active tool dictionary, and securely execute the Python function.³⁸ The resulting observation is then injected back into the model's context window, ensuring that the fragile local LLM remains securely tethered to the execution rails.³⁸

5. Hybrid Code Patterns: Node Wrappers and Execution Logistics

The technical synthesis of this architecture occurs at the boundary layer: the LangGraph node wrapper. Within LangGraph, nodes are standard Python functions that accept the current application state, execute arbitrary programmatic logic, and return a dictionary representing the state modifications to be merged.⁶ The complexity of the Lead Investigator arises from seamlessly orchestrating a full CrewAI lifecycle within the transient lifespan of this single function execution.¹²

5.1 Architecting the Wrapper Logic

The node wrapper serves as a contextual translation layer. It must extract relevant target parameters from the rigid LangGraph macro-state, dynamically instantiate the CrewAI components, and safely handle the resulting output.¹²

To maintain a clean separation of concerns and adhere to enterprise software engineering standards, the foundational configurations for the CrewAI agents and tasks should be decoupled from the execution logic, ideally stored in external YAML files.¹¹ This allows prompt engineers to iterate on the agent backstories and task descriptions without modifying the core Python orchestration code.¹¹

The execution sequence within the node wrapper unfolds through five distinct phases:

1. **State Ingestion and Pre-processing:** The LangGraph engine invokes the wrapper function, passing the current `JobApplicationState`.¹ The wrapper extracts the immediate objective—for example, popping the next target corporate entity from the `filtered_jobs` backlog list.¹
2. **Dynamic Component Instantiation:** The wrapper loads the external YAML configurations and instantiates the Agent object, binding the Lead Investigator role, backstory, and the specific suite of OSINT discovery tools.¹² Simultaneously, the Task object is constructed, embedding the target company name into the task description via format strings.¹²
3. **Strict Output Enforcement:** Crucially, the `output_pydantic` parameter is mapped to the Task definition, ensuring the language model is constrained to generate the structured B2B contact payload.²⁰
4. **Micro-Orchestration (kickoff):** The Crew object is assembled, integrating the agent and task into a sequential processing pipeline.¹² The `crew.kickoff()` method is invoked, transferring the execution thread entirely to the autonomous agent loop.¹²
5. **State Reintegration:** Upon the successful resolution of the task, the `CrewOutput` object is parsed. The rigorously validated Pydantic models representing the discovered contacts are extracted, formatted into the dictionary structure expected by LangGraph, and returned to the macro-orchestrator to be appended to the global state.¹⁹

5.2 Optimizing Concurrency and Batch Processing

In a production scenario focused on B2B lead generation, the system will rarely process a single target; it must navigate extensive lists of prospective companies simultaneously. Relying on a naive, sequential iteration loop to execute the CrewAI kickoff within the LangGraph node creates an unsustainable performance bottleneck, leaving computational resources idle while the agent waits for network I/O during web scraping or SMTP pinging.⁶

To achieve enterprise-grade throughput, the architecture must leverage asynchronous execution. CrewAI offers two divergent pathways for non-blocking operations: thread-based wrappers and native coroutines.³⁹ The `kickoff_async()` method wraps the standard synchronous execution within an `asyncio.to_thread` call, effectively farming the workload out to a thread pool executor.³⁹ While manageable for simple operations, this approach provides limited control over the underlying thread mechanics and can lead to resource exhaustion under heavy load.⁴¹

For highly concurrent macro-orchestration, the definitive best practice is utilizing the `akickoff()` method.³⁹ This approach implements true native `async/await` syntax throughout the entire execution chain, covering task processing, memory operations, and external knowledge retrieval.³⁹ By defining the LangGraph node wrapper as an asynchronous function (`async def lead_investigator_node(state):`), the framework can launch dozens of simultaneous CrewAI investigations.³⁹ When an individual agent initiates a web search, the asynchronous execution yields control back to the event loop, allowing other agents to continue processing, drastically reducing the overall latency of the pipeline.³⁹

However, developers must exercise stringent concurrency management when coupling asynchronous execution with localized infrastructure, such as Ollama inference endpoints.⁴¹ Bombarding a local GPU with unbounded parallel requests will inevitably result in Out-Of-Memory (OOM) faults or devastating inference degradation.⁴¹ In these local-compute scenarios, implementing a customized asynchronous Semaphore within the LangGraph node to strictly gate the maximum number of concurrent `akickoff()` instances ensures maximum throughput without violating the hardware's capacity.⁴¹

6. Advanced Error Handling, Fallbacks, and System Resilience

The true measure of a production-ready agentic system is its behavior in the face of inevitable failure. External web endpoints change their DOM structures, scraping APIs enforce rate limits, and language models sporadically return empty or heavily hallucinated payloads.¹⁷ Constructing the Lead Investigator node atop a hybrid architecture allows developers to deploy targeted defense mechanisms at multiple layers of the abstraction stack.¹

6.1 State Persistence and Graph-Level Retries

LangGraph's defining architectural advantage is its robust management of application state and cyclical fault recovery.⁴ In a traditional, monolithic multi-agent script, an unhandled exception thrown during the retrieval of a specific corporate website would crash the entire application, permanently deleting all previously aggregated lead data.⁶

By persisting the execution state through LangGraph's checkpoint mechanism (utilizing the InMemorySaver for local testing or persistent backends for production), the global application state is rigorously serialized and saved following the successful completion of each individual node.¹ Should the CrewAI micro-executor experience a fatal crash while investigating the fortieth company in a backlog, the graph execution halts safely.⁶ Developers can then debug the specific failure point, and effortlessly resume the application using the exact same session ID, picking up flawlessly from the thirty-ninth iteration.⁶

Furthermore, LangGraph eliminates the need to pollute the CrewAI execution logic with sprawling try-except blocks. The framework provides native, graph-level retry policies.⁴² Developers can configure the Lead Investigator node to automatically intercept transient network errors or specific API timeouts, instructing the orchestrator to dynamically re-invoke the node using configurable exponential backoff strategies.⁴² This creates a self-healing pipeline capable of weathering temporary infrastructure turbulence without human intervention.⁴²

6.2 Agent-Level Mitigation and LLM Fallback Routing

While LangGraph manages structural failures, the CrewAI layer must be fortified against inference-level anomalies. Even with optimized prompts, language models can occasionally enter looping states or generate invalid reasoning paths that bypass the Pydantic constraints.¹⁷ The primary mitigation strategy relies on the unified memory system inherent in CrewAI.⁴⁴ By ensuring the agent is configured to utilize this composite memory architecture—which blends semantic similarity, recency, and importance weighting—the agent maintains a coherent understanding of previously failed tool executions, significantly reducing the probability of repeating the exact same error.⁴⁴

The most critical resilience mechanism, however, is dynamic LLM fallback routing. When operating commercial APIs (e.g., OpenAI or Anthropic), service degradations are inevitable.⁴⁵ The hybrid architecture must guarantee operational continuity by gracefully degrading to alternative providers or localized models.

CrewAI leverages the LiteLLM library, enabling sophisticated routing protocols.⁴⁵ If the primary high-parameter model experiences an outage, the system can automatically reroute the prompt to a local Ollama instance running a smaller model like Llama 3.⁴⁵ Configuring this fallback cascade requires careful attention to specific model parameters. Because different models support disparate API parameters (for instance, certain models reject specific stop sequences or temperature formats), configuring the secondary LLM object with attributes such as `drop_params=True` and `additional_drop_params=["stop"]` ensures that incompatible

arguments are safely stripped from the payload during the transition, preventing the fallback request from failing due to schema validation errors.⁴⁵

7. Human-in-the-Loop (HITL) Execution and Escalation Routing

Despite the implementation of rigorous constraints and fallback strategies, autonomous OSINT gathering remains an inherently probabilistic endeavor. B2B contact discovery frequently produces ambiguous intelligence; an agent might confidently extract the email address of a "VP of Employee Success" when the explicit objective required a "Director of Machine Learning Operations." Permitting an autonomous system to blindly push unverified or low-confidence data into downstream CRM pipelines or automated outreach sequences presents unacceptable business risks. The architecture must incorporate deterministic oversight.¹

LangGraph fundamentally resolves this challenge by providing native, first-class support for Human-in-the-Loop (HITL) interventions, a capability that requires extensive custom engineering to replicate in pure multi-agent frameworks.⁵ The macro-orchestrator acts as an automated auditor, evaluating the metadata appended to the state by the CrewAI micro-executor.¹

As the HREmailLead objects are merged into the LangGraph state, a dedicated routing function assesses the confidence scores calculated by the language model.¹ This router dictates the conditional edges of the graph based on predefined heuristic thresholds:

- If the aggregated contacts meet the required confidence threshold (e.g., `confidence_score >= 0.95`), the LangGraph router automatically directs the workflow to the subsequent automated node, perhaps initiating a personalized outreach sequence.¹
- If the agent returns an empty dataset, signals an inability to penetrate the corporate domain, or produces contacts with sub-optimal confidence scores (`confidence_score < 0.95`), the router triggers an `escalation_needed` flag.¹

When the escalation flag is raised, the workflow traverses a conditional edge to a dedicated HITL node.¹ This node utilizes LangGraph's native `interrupt()` function.¹² Invoking this method instantly pauses the execution of the entire graph, safely persisting the current application state to the configured checkpoint database, and actively yielding control back to the hosting environment.¹

This suspension allows the enterprise application (such as a FastAPI backend feeding a React dashboard) to present the ambiguous findings, the agent's analytical reasoning trace, and the specific error logs to a human operator for review.¹ The operator can manually verify the contact data, reject hallucinated entries, or provide specific guidance to the system.

Following the manual review, the application resumes the autonomous workflow by submitting a `Command(resume="Operator Input Payload")` back to the suspended graph thread.¹² The

graph integrates this external feedback directly into the state schema, allowing the workflow to proceed safely to the termination node or dynamically loop back to the CrewAI node armed with the newly provided human context.¹ This cyclical, interrupt-driven pattern guarantees that the generative power of the autonomous agents remains strictly tethered to human verification protocols.

8. Synthesized Directives for Code Generation

The fusion of LangGraph and CrewAI is not merely a theoretical exercise; it is a highly pragmatic architectural pattern that demands meticulous execution.⁶ For a downstream language model to successfully synthesize the Python code for this Lead Investigator node, it must strictly adhere to the guidelines detailed throughout this document.

The generated implementation must utilize a robust TypedDict or Pydantic BaseModel for the macro-graph state, seamlessly accommodating the injection of the CrewOutput.⁷ The CrewAI components must be defined with deeply contextualized personas, employing the 80/20 rule to prioritize exacting task definitions, and leveraging output_pydantic to enforce structured JSON extraction without the latency of secondary LLM parsing.¹⁵

If the implementation specifies localized inference via Ollama, the generative code must reflect hyper-constrained configurations, including zero-temperature sampling and rigorously molded system prompts designed to eliminate infinite tool-calling loops and hallucinated schemas.³¹ The code must demonstrate asynchronous scalability using akickoff(), guarded by appropriate thread management to protect local hardware.³⁹ Finally, the architectural flow must encompass resilient fault boundaries, demonstrating LangGraph's checkpoint persistence and explicitly defining the conditional edges required for Human-in-the-Loop interventions via the interrupt() mechanism.¹ By adhering to these specifications, the resulting system transcends brittle scripting, establishing a scalable, fault-tolerant, and profoundly capable enterprise investigative asset.

Works cited

1. research-AI Engineer Career Strategy with LangGraph and CrewAI.docx
2. The Ultimate Guide to Contact Discovery and Building B2B Lists - DealSignal, accessed May 7, 2026, <https://www.dealsignal.com/contact-discovery-processes/>
3. Chat GPT Prompt Engineering for Executive Research for B2B Sales - Reddit, accessed May 7, 2026, https://www.reddit.com/r/sales/comments/1aq4a5m/chat_gpt_prompt_engineering_for_executive/
4. In-Depth Comparison: Workflow Control with LangGraph and CrewAI - DEV Community, accessed May 7, 2026, <https://dev.to/rosidotidev/in-depth-comparison-workflow-control-with-langgraph-and-crewai-ae3>
5. LangGraph vs CrewAI vs AutoGen: Which AI Agent Framework Should Your

- Enterprise Use in 2026? - Towards AI, accessed May 7, 2026,
<https://pub.towardsai.net/langgraph-vs-crewai-vs-autogen-which-ai-agent-framework-should-your-enterprise-use-in-2026-3a9ebb407b09>
6. How to Build a Multi-Agent AI System with LangGraph, MCP, and A2A [Full Book], accessed May 7, 2026,
<https://www.freecodecamp.org/news/how-to-build-a-multi-agent-ai-system-with-langgraph-mcp-and-a2a-full-book/>
 7. Moving from LangGraph to CrewAI: A Practical Guide for Engineers, accessed May 7, 2026,
<https://docs.crewai.com/en/guides/migration/migrating-from-langgraph>
 8. Comparing AI agent frameworks: CrewAI, LangGraph, and BeeAI - IBM Developer, accessed May 7, 2026,
<https://developer.ibm.com/articles/awb-comparing-ai-agent-frameworks-crewai-langgraph-and-beeai/>
 9. What is crewAI? - IBM, accessed May 7, 2026,
<https://www.ibm.com/think/topics/crew-ai>
 10. Agents - CrewAI Documentation, accessed May 7, 2026,
<https://docs.crewai.com/en/concepts/agents>
 11. Building Multi-Agent Systems With CrewAI - A Comprehensive Tutorial - Firecrawl, accessed May 7, 2026,
<https://www.firecrawl.dev/blog/crewai-multi-agent-systems-tutorial>
 12. Combining LangGraph and CrewAI. In my previous blog, I shared my... | by Mayada Khatib | Medium, accessed May 7, 2026,
<https://medium.com/@mayadakhatib/combining-langgraph-and-crewai-bf38c719ab27>
 13. langgraph-with-crewai/langgraph.ipynb at main · Ionio-io/langgraph-with-crewai - GitHub, accessed May 7, 2026,
<https://github.com/Ionio-io/langgraph-with-crewai/blob/main/langgraph.ipynb>
 14. Ionio-io/langgraph-with-crewai - GitHub, accessed May 7, 2026,
<https://github.com/Ionio-io/langgraph-with-crewai>
 15. Bring your own agent - CrewAI Documentation, accessed May 7, 2026,
<https://docs.crewai.com/en/learn/bring-your-own-agent>
 16. Tasks - CrewAI Documentation, accessed May 7, 2026,
<https://docs.crewai.com/en/concepts/tasks>
 17. Handling LLM Errors in Hierarchical CrewAI Process with Callbacks, accessed May 7, 2026,
<https://community.crewai.com/t/handling-llm-errors-in-hierarchical-crewai-process-with-callbacks/4694>
 18. Constraining LLMs with Structured Output: Ollama, Qwen3 & Python or Go - Medium, accessed May 7, 2026,
<https://medium.com/@rosgluk/constraining-llms-with-structured-output-ollama-qwen3-python-or-go-2f56ff41d720>
 19. Structured output for stateGraph - LangGraph - LangChain Forum, accessed May 7, 2026,
<https://forum.langchain.com/t/structured-output-for-stategraph/2726>

20. How to use Output Pydantic in a Task? - crewAI+ Help Center, accessed May 7, 2026, <https://help.crewai.com/how-to-use-output-pydantic-in-a-task>
21. How to get structured output using pydantic defined model? · crewAI Inc crewAI · Discussion #1436 - GitHub, accessed May 7, 2026, <https://github.com/crewAIInc/crewAI/discussions/1436>
22. Customize Agents - CrewAI Documentation, accessed May 7, 2026, <https://docs.crewai.com/en/learn/customizing-agents>
23. Crafting Effective Agents - CrewAI Documentation, accessed May 7, 2026, <https://docs.crewai.com/en/guides/agents/crafting-effective-agents>
24. Build a Lead Generation AI tool with Multi AI Agents - YouTube, accessed May 7, 2026, https://www.youtube.com/watch?v=8ISy4imaU_8
25. What is Prompt Engineering? Tips & Best Practices | Salesforce EU, accessed May 7, 2026, <https://www.salesforce.com/eu/artificial-intelligence/prompt-engineering/>
26. Context Engineering Deep Dive: Building a Deep Research Agent, accessed May 7, 2026, <https://www.promptingguide.ai/agents/context-engineering-deep-dive>
27. Mastering prompt engineering for B2B sales: six tips for prompts, accessed May 7, 2026, <https://mercuri.net/insights/mastering-prompt-engineering-for-b2b-sales-six-building-blocks-of-an-effective-prompt/>
28. Effective context engineering for AI agents - Anthropic, accessed May 7, 2026, <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
29. Agent Capabilities - CrewAI Documentation, accessed May 7, 2026, <https://docs.crewai.com/en/concepts/agent-capabilities>
30. Tool Calling with Local LLMs: A Practical Evaluation - Docker, accessed May 7, 2026, <https://www.docker.com/blog/local-llm-tool-calling-a-practical-evaluation/>
31. How to do proper function calling on Ollama models - Reddit, accessed May 7, 2026, https://www.reddit.com/r/ollama/comments/1ioyxkm/how_to_do_proper_function_calling_on_ollama_models/
32. Tool calling chain with local Ollama models (7b/14b) - 2nd tool never executed - Questions, accessed May 7, 2026, <https://community.n8n.io/t/tool-calling-chain-with-local-ollama-models-7b-14b-2nd-tool-never-executed/280320>
33. getting llama3 to produce proper json through ollama : r/LocalLLaMA - Reddit, accessed May 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1d3x3m5/getting_llama3_to_produce_proper_json_through/
34. Qwen3 - Tool Call hallucination · Issue #11135 · ollama/ollama - GitHub, accessed May 7, 2026, <https://github.com/ollama/ollama/issues/11135>
35. Recommendations for running custom tools with local ollama models (having function calling capabilities) - CrewAI Community Support, accessed May 7, 2026, <https://community.crewai.com/t/recommendations-for-running-custom-tools-with-local-ollama-models-having-function-calling-capabilities/5777>
36. Tool calling - Ollama's documentation, accessed May 7, 2026,

- <https://docs.ollama.com/capabilities/tool-calling>
37. erukude/multiagent-orchestrator - Ollama, accessed May 7, 2026, <https://ollama.com/erukude/multiagent-orchestrator>
 38. Achieving Tool Calling Functionality in LLMs Using Only Prompt Engineering Without Fine-Tuning Citation: Authors. Title. Pages.... DOI:000000/11111. - arXiv, accessed May 7, 2026, <https://arxiv.org/html/2407.04997v1>
 39. Kickoff Crew Asynchronously - CrewAI Documentation, accessed May 7, 2026, <https://docs.crewai.com/en/learn/kickoff-async>
 40. Technical Comparison of AutoGen, CrewAI, LangGraph, and OpenAI Swarm, accessed May 7, 2026, <https://ai.plainenglish.io/technical-comparison-of-autogen-crewai-langgraph-and-openai-swarm-1e4e9571d725>
 41. Batch Processing Best Practice - CrewAI Community Support, accessed May 7, 2026, <https://community.crewai.com/t/batch-processing-best-practice/6991>
 42. A Beginner's Guide to Handling Errors in LangGraph with Retry Policies - DEV Community, accessed May 7, 2026, <https://dev.to/aiengineering/a-beginners-guide-to-handling-errors-in-langgraph-with-retry-policies-h22>
 43. Advanced Error Handling Strategies in LangGraph Applications - Sparkco, accessed May 7, 2026, <https://sparkco.ai/blog/advanced-error-handling-strategies-in-langgraph-applications>
 44. accessed May 7, 2026, <https://docs.crewai.com/llms-full.txt>
 45. "Fallback" LLM Configuration - LLMs - CrewAI, accessed May 7, 2026, <https://community.crewai.com/t/fallback-llm-configuration/6190>
 46. Streaming responses with tool calling · Ollama Blog, accessed May 7, 2026, <https://ollama.com/blog/streaming-tool>